

企业级应用软件开发与开发

期末大作业报告

项目名称：TravelMind 企业差旅多智能体助手

方向：方向一：Agentic AI 原生开发

字段	内容
课程名称	企业级应用软件开发与开发
课程代码	50120224001 / CS599
学号	2025303057
姓名	任锦炫
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

- 一、选题背景与设计思想
- 二、Specs 规格文档
- 三、系统架构与设计
- 四、关键实现与代码展示
- 五、测试与评估
- 六、系统升级与扩展
- 七、课程总结

一、选题背景与设计思想

企业差旅是典型的多信息源、多步骤业务场景。员工在出差前需要同时理解企业制度、报销标准、实时交通天气、目的地信息、个人偏好和历史行程。传统关键词问答只能回答单点问题，普通流程系统又难以理解自然语言中的模糊表达，因此需要一个能理解上下文、调用工具、规划步骤并维护记忆的 Agentic AI 系统。

TravelMind 的设计思想是把差旅咨询和行程规划拆解为可编排的能力单元：意图识别智能体负责理解用户需求，协调器负责生成执行顺序和并行批次，子智能体分别负责记忆查询、偏好管理、事项收集、RAG 知识库问答、联网信息查询和行程规划。

技术路线

- 使用 SDD 方法建立 Product Spec、Architecture Spec 和 API Spec，将需求、结构和接口前置。
- 使用 AgentScope 与 OpenAI 兼容模型接口构建多智能体系统。
- 以 Skill 插件形式组织子 Agent，让新增能力不影响主流程。
- 使用本地 BGE embedding 模型与 Milvus Lite 实现企业差旅知识库 RAG。
- 使用短期窗口与长期持久化记忆支持跨会话个性化。
- 加入重试、熔断和健康检查，提高外部模型服务不稳定时的可用性。

二、Specs 规格文档

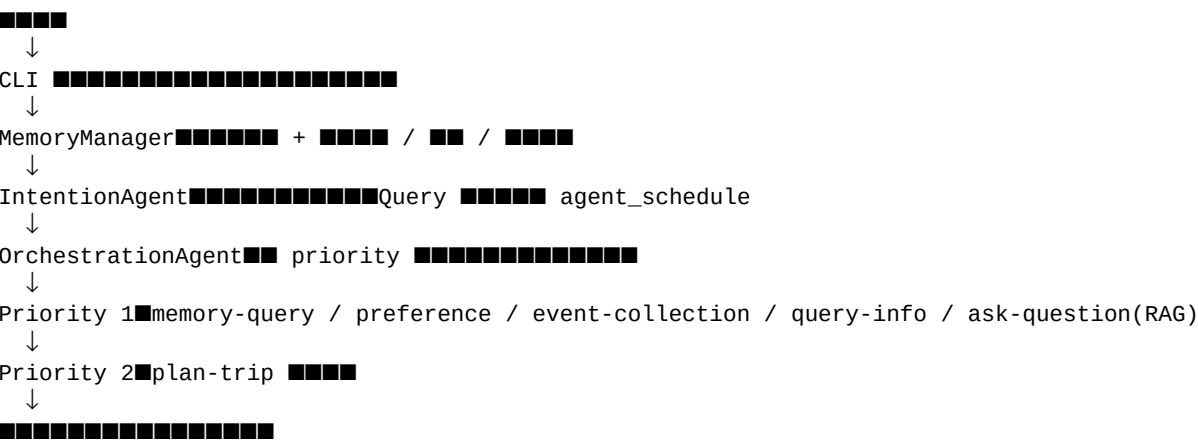
本项目采用 SDD（规格驱动开发）组织工程闭环。规格文档存放在 docs/specs 目录下，分别约束产品需求、系统架构和内部接口协议。

规格文档	核心内容	对应代码
Product Spec	用户场景、功能需求、非功能需求、验收标准	README、CLI、Agent 能力边界
Architecture Spec	模块关系、Agent 交互流程、数据流、扩展点	agents/、context/、utils/、.claude/skills/
API Spec	环境变量、IntentionAgent 输出 JSON、OrchestrationAgent 输入协议、Skill 目录约定	config.py、intention_agent.py、orchestration_agent.py、skill.py

规格文档直接约束代码实现：IntentionAgent 必须输出可执行的 agent_schedule；OrchestrationAgent 必须根据 priority 批量调度；Skill 插件必须通过 SKILL.md 暴露能力说明并在 script/agent.py 中实现执行逻辑。

三、系统架构与设计

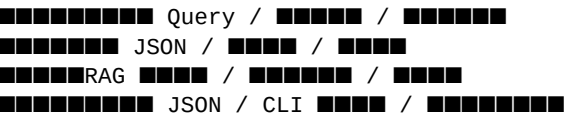
核心架构图



Agent 交互流程

1. CLI
2. CLI
3. IntentionAgent
4. OrchestrationAgent
5. Agent
6. Agent

数据流设计



架构上，CLI 只负责交互入口；记忆管理器提供上下文；意图识别智能体把自然语言变成结构化计划；协调器负责执行策略；Skill Agent 负责工具化能力。这样可以在不改动主流程的情况下新增制度问答、审批审核、报销校验等企业级能力。

四、关键实现与代码展示

实现点	文件	说明
Agent 核心循环	cli.py	读取用户输入、合并记忆、调用意图识别、调度执行、输出结果
意图识别	agents/intention_agent.py	生成 reasoning、intents、key_entities、rewritten_query、
多智能体编排	agents/orchestration_agent.py	按优先级分组，同优先级使用 asyncio.gather 并行执行
工具定义	.claude/skills/*/SKILL.md	声明每个子 Agent 的能力、触发条件和执行说明
RAG 工具	.claude/skills/ask-question/skill.py	向量检索企业差旅制度并生成回答
安全配置	config.py / .env.example	API Key 从环境变量读取，避免硬编码

调度计划示例

```
{
  "agent_schedule": [
    {"agent_name": "event_collection", "priority": 1},
    {"agent_name": "rag_knowledge", "priority": 1},
    {"agent_name": "itinerary_planning", "priority": 2}
  ]
}
```

AI IDE 使用记录：项目开发过程使用 AI 辅助完成需求拆解、规格文档整理、Prompt 结构优化、README 重写、密钥安全修复和测试问题定位。最终提交前可在答辩 PPT 或 Demo 中补充 Trae CN / AI IDE 的操作截图或录屏。

五、测试与评估

测试材料位于 tests/ 与 tests/results/。由于系统依赖外部 LLM 和联网搜索，测试分为可离线验证的结构测试、需要 API Key 的端到端测试，以及用于报告展示的 Demo 截图/录屏。

测试类型	测试目标	证据位置
记忆系统测试	验证短期记忆、长期偏好、历史行程和对话记录	tests/test_memory_system.py
意图识别测试	验证多意图识别、实体抽取和 JSON 输出	tests/test_intention_agent.py
RAG 测试	验证企业差旅制度问答与来源检索	tests/test_rag_agent.py
编排测试	验证 priority 分组、并行执行和结果聚合	tests/test_orchestration.py
问答评估	保存多轮问答与 Demo 结果	tests/results/*.md
评估指标	目标值	说明
意图识别可用率	90%+	典型差旅表达是否生成正确调度计划
RAG 命中率	85%+	制度问题是否检索到相关文档
偏好持久化成功率	95%+	跨会话能否读取用户偏好
端到端响应成功率	80%+	受 LLM API 与联网搜索稳定性影响

Demo 展示建议：准备三组输入，分别覆盖企业制度问答、偏好记忆、完整行程规划。若现场 API 异常，可使用 tests/results 中的历史问答记录和本地录屏作为保底材料。

六、系统升级与扩展

- 将 CLI 升级为 Web UI、飞书或企业微信机器人入口，让系统进入真实企业工作流。
- 引入 LangGraph 状态机，使流程恢复、失败分支、人工确认和长任务执行更清晰。
- 加入 OpenTelemetry、LangSmith 或本地 tracing，记录每次 Agent 调用耗时、输入输出和失败原因。
- 将长期记忆从 JSON 文件升级为 PostgreSQL 或 Redis + PostgreSQL 组合。

- 增加权限控制，区分普通员工、财务和管理员可见的制度内容。
- 加入报销单据识别、预算校验和审批路由，进一步贴近企业级应用软件场景。

七、课程总结

通过本项目，我对“写一个模型调用脚本”和“构建一个 Agentic AI 系统”的差异有了更具体的认识。后者不仅关注单次回答质量，还要关注规格约束、状态管理、工具边界、失败恢复、记忆持久化和可评估性。

SDD 方法让我先定义输出协议和验收标准，再实现 Agent 逻辑，减少了 Prompt 和代码之间互相漂移的问题。多智能体编排也让我意识到，Agent 系统的关键不是堆叠更多模型调用，而是把任务拆成可观察、可替换、可测试的能力单元。

课程层面的收获是工程思维的转变：从“完成一个功能”转向“设计一个可维护、可验证、可扩展的智能系统”。后续如果继续完善，我会优先补齐可观测性、线上部署和更严格的 Agent 行为评估。

参考资料与引用声明

- AgentScope 官方文档与开源项目。
- Milvus / PyMilvus 官方文档。
- BAAI BGE 中文 embedding 模型说明。
- DDGS / DuckDuckGo 搜索相关开源库。
- 若项目中存在来源于外部项目的代码或资料，最终提交前应补充具体来源、链接和许可证，避免学术诚信风险。